GUIA DE PROJETO • NEXT.JS • OUTUBRO DE 2026

TERRA EM MOVIMENTO

Guia de construção e publicação

Planejamento resumido para transformar o protótipo do Stitch em uma experiência web interativa, testada e publicada na Vercel.

OBJETIVO

Globo 3D + dados de países + quatro páginas consistentes

Uma API externa, dados geográficos locais e cache com prazo limitado.

Next.js | TypeScript | Tailwind CSS | React Globe.gl | GSAP | GitHub → Vercel

Base analisada: ZIP de exemplo da sala de aula

23 OUT 2026

1. O produto e a experiência

A tela do Stitch é a referência visual. O objetivo do MVP é entregar uma experiência rápida, bonita e fácil de explorar durante o evento.

Identidade visual

Tema escuro: fundo #070B14, superfícies #0E1625, ciano #38BDF8, turquesa #2DD4BF, texto claro e bordas discretas. Use a mesma tipografia, espaçamento, cabeçalho e rodapé em todas as páginas. O globo deve ser o elemento dominante da home.

As quatro páginas

ROTA	CONTEÚDO	COMPORTAMENTO
/ — Mundo	Globo 3D, busca, filtros por região, pequenos marcadores e painel de país selecionado.	Hover destaca e mostra tooltip; clique fixa os detalhes num painel flutuante.
/paises — Países	Cards com bandeira, nome, continente, capital e população; pesquisa e filtros.	Renderizar cards com `countries.map()`; filtrar a lista já carregada no cliente.
/projeto — Sobre o projeto	Objetivo, como funciona, APIs, tecnologias e desenvolvimento assistido por IA.	Conteúdo estático; sem chamadas à API.
/sobre — Sobre mim	Apresentação editável, jornada, tecnologias e links reais para GitHub/LinkedIn.	Conteúdo estático; sem inventar experiência ou links.

Decisão de interação recomendada

Tooltip + painel	Ao passar o mouse, exiba apenas o nome e a bandeira. Ao clicar, mostre capital, população, idiomas e moeda num painel lateral/flutuante. Na página Países, o clique no card abre o mesmo painel ou detalhes equivalentes. Isso mantém a visita rápida e evita uma rota extra só para detalhes.

Escopo essencial: globo, países selecionáveis, cards com busca/filtros, quatro páginas e publicação confiável. Deixe terremotos e indicadores econômicos para depois; não são necessários para provar o conceito.

2. API, globo e cache

A arquitetura mínima usa uma API externa de países e um arquivo geográfico local. O globo é uma biblioteca de visualização, não uma API.

Fontes e responsabilidades

PEÇA	FUNÇÃO	REQUISIÇÕES / OBSERVAÇÕES
REST Countries v5 [1]	Nome, bandeira, código ISO, capital, continente, população, idiomas e moedas.	Plano gratuito documentado: 1.000 requisições/mês; até 100 países por chamada. Requer token Bearer. Resposta pode ser cacheada por no máximo 3 dias [1][2].
GeoJSON local [3]	Contornos/polígonos que o globo usa para desenhar e detectar o país clicado.	Arquivo em `public/data/countries.geojson`; não é uma API. Confira licença e códigos antes de usar.
React Globe.gl [4]	Renderiza globo 3D, polígonos, hover e clique.	Biblioteca WebGL/Three.js. Precisa rodar no navegador; importe o componente como Client Component/dinâmico se necessário.
GSAP [5]	Animações de entrada, cards, painéis e detalhes que não pertencem à esfera 3D.	Use com parcimônia; integrar ao React com `useGSAP` ou cleanup adequado.

Como manter o limite sob controle

- Busque os países no servidor Next.js, em lotes de até 100 (por exemplo, offsets 0, 100 e 200, parando quando a paginação indicar que não há mais resultados).
- A busca, a ordenação e os filtros por continente são locais sobre o conjunto carregado; não faça uma requisição por card nem a cada tecla digitada.
- Configure cache com validade explícita de até 259.200 segundos (3 dias), respeitando os Termos atuais da REST Countries [2]. Não armazene nem sirva cópia das respostas fora dessa janela.
- A contagem real depende do número de páginas retornadas e da invalidação do cache. O cache não é armazenamento permanente.

Fluxo Navegador → componentes Next.js → módulo de dados no servidor → REST Countries. Em paralelo, o globo lê GeoJSON local. Ao clicar em um polígono, o código ISO liga a geometria aos dados do país.

Proteção da chave

Guarde `REST_COUNTRIES_API_KEY` em `.env.local` no desenvolvimento e em Vercel → Settings → Environment Variables na produção. Use-a apenas em código do servidor. Nunca use prefixo `NEXT_PUBLIC_`, não comite o arquivo e não coloque a chave no componente do globo.

3. Arquitetura Next.js com base no ZIP

O arquivo enviado é um esqueleto de Next.js, não o dashboard pronto. Aproveite sua organização em vez de começar do zero.

O que já existe no ZIP

Next.js 16.3.8, React 19.2.8, TypeScript e Tailwind CSS 4 aparecem no package.json. Há `src/app/layout.tsx`, `src/app/page.tsx`, o CSS global com `@import "tailwindcss"`, scripts npm e uma rota exemplo em `src/app/(pages)/produtos/page.tsx`. O exemplo é mínimo (Hello world/Produto); páginas, componentes, API e globo ainda precisam ser implementados.

Estrutura proposta

```
src/
  app/
    layout.tsx                # shell global, fontes, metadata
    page.tsx                  # / (Mundo)
    (pages)/                  # grupo; nao aparece na URL
      paises/page.tsx          # /paises
      projeto/page.tsx         # /projeto
      sobre/page.tsx           # /sobre
    not-found.tsx             # 404
    error.tsx                 # erro de segmento
    global-error.tsx          # erro global, se necessario
    429/page.tsx              # tela explicita opcional
  components/
    Header.tsx
    Footer.tsx
    GlobeScene.tsx
    CountryCard.tsx
    CountryDetails.tsx
  lib/
    countries.ts              # fetch no servidor + normalizacao
  types/
    country.ts                # tipo Country padronizado
public/
  data/countries.geojson
```

Decisões de arquitetura

- layout.tsx: header e footer compartilhados; definir `lang="pt-BR"`, título e descrição do site.
- Server Components por padrão: buscar dados no servidor e passar uma lista reduzida aos componentes interativos.
- Globo como Client Component: depende do navegador/WebGL; use importação dinâmica se a biblioteca não for compatível com SSR.
- Normalizar os dados: converter a resposta da API para um tipo único (código ISO, nome, capital, continente, população, bandeira, idiomas e moedas). Campos podem estar ausentes.
- Tailwind CSS: o ZIP já tem Tailwind 4; preserve o setup inicial e mude tokens no CSS global para o tema escuro.
- GSAP: anima elementos da interface, não substitui o motor 3D. Limpe animações quando o componente desmontar.

Atenção à documentação — O AGENTS.md do ZIP avisa que a versão de Next.js instalada pode ter diferenças. Antes de pedir código a uma IA, confira as guias locais em node_modules/next/dist/docs/ e confirme a sintaxe na documentação da versão instalada.

4. Passo a passo de construção

Implemente em camadas. Só avance quando a etapa atual estiver funcionando localmente.

ETAPA	O QUE FAZER / CONDIÇÃO DE SAÍDA
01 · Preparar a base	Descompactar/clonar o ZIP, instalar dependências com npm ci, executar npm run dev e conferir a aplicação inicial.
02 · Definir o design system	Aplicar paleta escura do Stitch, tipografia, Header, Footer, botão, input, chip de filtro e estilos responsivos.
03 · Criar páginas e rotas	Montar Mundo, Países, Sobre o projeto e Sobre mim; links ativos no header e conteúdo estático onde apropriado.
04 · Provar o globo isolado	Instalar react-globe.gl, carregar GeoJSON local, configurar atmosfera/câmera e testar polígonos e clique. Manter esse componente separado.
05 · Integrar REST Countries	Criar a variável de ambiente, função de servidor, paginação em lotes, seleção dos campos e cache de até três dias. Normalizar a resposta para Country.
06 · Unir os dados	Relacionar geometria e dados por código ISO. Hover mostra tooltip; clique abre painel. Conferir países sem capital/idiomas ou dados incompletos.
07 · Construir os cards	Renderizar com countries.map(); busca, filtro e ordenação usam os dados carregados. Reutilizar o mesmo componente ou painel de detalhes.
08 · Animações e acabamento	Adicionar GSAP aos elementos da interface, estados de loading/erro, acessibilidade, responsividade e animação reduzida quando o usuário prefere menos movimento.
09 · Erros e conteúdo estático	Criar 404 e telas de erro, terminar Sobre o projeto/Sobre mim e revisar textos, links e metadados.
10 · Congelar funcionalidades	Corrigir erros, verificar build de produção e implantar. Evitar novas funcionalidades depois do período final de testes.

Ordem de prioridade para 7 de novembro

Must-have — Globo utilizável → clique em país → cards com dados reais → navegação pelas quatro páginas → publicação estável. Terremotos, gráficos econômicos e efeitos decorativos extras ficam para depois, se houver tempo.

5. Checklist antes de publicar

Execute os testes abaixo antes do primeiro deploy e novamente antes de congelar a versão do evento.

Build e código

- ☐ npm ci conclui sem erro.
- ☐ npm run lint conclui sem erro.
- ☐ npx tsc --noEmit conclui sem erro (se scripts/tsconfig não tiverem sido alterados).
- ☐ npm run build conclui com sucesso.
- ☐ Não há erros nem warnings críticos no console do navegador.

Páginas e navegação

- ☐ Mundo, Países, Sobre o projeto e Sobre mim abrem pelo header.
- ☐ Recarregar diretamente /paises, /projeto e /sobre funciona.
- ☐ Header/footer consistentes; o link da página atual tem destaque.
- ☐ 404 aparece ao abrir uma rota inexistente.

API, cache e dados

- ☐ Chave válida retorna dados; chave ausente/inválida produz mensagem útil e não expõe segredo.
- ☐ Paginação percorre todos os lotes e termina sem duplicar ou pular países.
- ☐ Busca/filtros/sort não fazem uma chamada à API por interação.
- ☐ Capital, idiomas, moedas, bandeira e população ausentes não quebram os cards.
- ☐ Tratar timeout, resposta não-JSON, erro 5xx e HTTP 429 (limite atingido).
- ☐ Confirmar no servidor o cache e a validade máxima de três dias exigida pelos termos da API.

Globo, animações e experiência

- ☐ GeoJSON desenha países; clicar em Brasil/Japão abre os dados corretos.
- ☐ O hover funciona e não cobre controles essenciais; clique é alternativa ao hover.
- ☐ GSAP não duplica animações ao navegar/montar de novo componentes.
- ☐ Testar em tela 1440×1000 e em janela menor; textos e painel não se sobrepõem.
- ☐ Testar mouse, teclado, foco visível, contraste e preferência por movimento reduzido.
- ☐ Testar no link de Preview da Vercel, não apenas em localhost.

6. GitHub → Vercel: publicação automática

Depois que o repositório estiver conectado, um push na branch de produção pode disparar um novo deploy automaticamente [6].

Configuração inicial (uma vez)

1. Criar um repositório no GitHub e enviar o projeto (não enviar .env.local).

2. Entrar na Vercel, escolher Add New → Project e importar o repositório GitHub.

3. Confirmar framework Next.js e diretório raiz correto; como o ZIP usa src/, a raiz continua sendo a pasta que contém package.json.

4. Em Settings → Environment Variables, cadastrar REST_COUNTRIES_API_KEY para Production e Preview (e Development se usar a integração local).

5. Definir main como Production Branch e fazer o primeiro deploy. As variáveis novas só entram nas implantações seguintes; redeploy após adicioná-las [7].

Fluxo recomendado no dia a dia

1. Trabalhar	2. Validar	3. Publicar
Criar branch para uma tarefa; codificar em partes pequenas.	Rodar lint, TypeScript e build local; abrir Preview da Vercel e testar.	Criar PR e mesclar na main depois de validar; a Vercel implanta a versão de produção.

Comandos iniciais (se o ZIP ainda não for um repositório Git)

```
git init
git add .
git commit -m "feat: inicio do Terra em Movimento"
git branch -M main
git remote add origin https://github.com/SEU-USUARIO/SEU-REPO.git
git push -u origin main
```

Atualizações seguintes

```
git add .
git commit -m "feat: adiciona cards de paises"
git push
```

Atenção Se o push for direto para main, ele pode atualizar a produção imediatamente. Para reduzir o risco de quebrar o site, use branch/PR e valide o Preview antes de mesclar. Cada push em uma branch comum gera Preview; a branch configurada como produção gera Production Deployment [6].

Antes do evento: deixe a versão principal estável até 6 de novembro. No dia 6, teste o endereço público, carregue a página Mundo, valide a chave e o cache e evite alterações grandes.

7. Erros, IA e preparação para o evento

Use as IAs para acelerar o trabalho, mas mantenha responsabilidade sobre integração, testes e decisões técnicas.

Tratamento das páginas de erro

ARQUIVO / ROTA	USO	OBSERVAÇÃO
`src/app/not-found.tsx`	Tela 404 para rota não encontrada.	Botão claro para voltar ao Mundo; preservar header/footer se apropriado.
`src/app/error.tsx`	Interface de recuperação para erros inesperados do segmento.	Inclui opção de tentar novamente quando aplicável. É uma UI boundary, não uma regra geral para forçar status HTTP 500.
`src/app/global-error.tsx`	Erros no layout raiz, se necessário.	Precisa fornecer a estrutura html/body conforme as convenções do Next.js.
`/429` **opcional**	Tela visual para limite temporário.	Criar a página não trata automaticamente respostas 429. O código que consulta a API deve detectar status e exibir a tela/estado certo.

Como dividir as tarefas com Copilot e outras IAs

- Delegue tarefas separadas: layout e páginas estáticas; componente de card; função de dados; globo; estados de erro. Evite pedir a aplicação inteira num único prompt.
- Antes de aceitar código, peça explicação das dependências, fluxo de dados, estados de falha e como testar. Leia os arquivos modificados e compare com a documentação da versão instalada.
- Você mantém como prioridade pessoal o globo, o vínculo entre GeoJSON e dados, o cache, a chave e o fluxo de deploy. Mesmo que a IA gere a implementação, você precisa conseguir explicar essas decisões.
- Faça um commit por etapa concluída; se uma tarefa gerada pela IA quebrar algo, fica simples revisar o diff ou reverter.

Agenda sugerida até 7 de novembro

23–25/10	26–29/10	30/10–02/11	03–06/11
Base, design system e rotas.	Globo e API; clique/tooltip e cards.	GSAP, páginas estáticas, acessibilidade e erros.	Lint, build, deploy preview/produção, correções e freeze.

Critério de pronto

No computador do evento, uma pessoa abre a home, entende o globo sem explicação longa, seleciona um país, vê detalhes, navega para Países e consegue retornar ao início. A aplicação não expõe chave, não mostra erros no console e tem um estado claro se a API falhar.

8. Fontes oficiais e referências

Links consultados em 23 de outubro de 2026. Confira a documentação da versão instalada antes de copiar código gerado por IA.

[1] REST Countries — Countries API v5

https://restcountries.com/docs/countries — Campos, autenticação Bearer, paginação, limites por consulta, JSON e uso da API.

[2] REST Countries — Termos de serviço

https://restcountries.com/legal/terms-of-service — Limite de cache/retensão de até três dias e restrições de armazenamento de respostas.

[3] Geo Countries dataset

https://github.com/datasets/geo-countries — GeoJSON de países; verificar a licença e a versão do arquivo usado.

[4] react-globe.gl — README e API

https://github.com/vasturiano/react-globe.gl — Renderização 3D, polígonos GeoJSON, eventos de hover/click.

[5] GSAP — instalação e documentação

https://gsap.com/docs/v3/Installation/ — Instalação, módulos e uso em aplicações React.

[6] Vercel — Git deployments

https://vercel.com/docs/git — Deploys de Preview e Produção disparados por pushes/merges.

[7] Vercel — Environment Variables

https://vercel.com/docs/environment-variables — Configuração separada por ambiente e aplicação nas próximas implantações.

[8] Next.js — convenções de arquivos

https://nextjs.org/docs/app/api-reference/file-conventions — layout.tsx, page.tsx, error.tsx, not-found.tsx, route groups e Route Handlers.

[9] Next.js — revalidação de dados

https://nextjs.org/docs/app/getting-started/revalidating — Estratégias de cache e revalidação de dados. Confirmar o modelo compatível com a configuração do ZIP.

[10] Tailwind CSS — Next.js

https://tailwindcss.com/docs/installation/framework-guides/nextjs — Integração de Tailwind CSS com Next.js.

[11] Vercel — deploy Next.js

https://vercel.com/frameworks/nextjs — Implantação Next.js, cache, renderização e integração contínua.

Resumo de decisões

Stack: Next.js + TypeScript + Tailwind CSS + react-globe.gl + GSAP. Dados: uma API de países (REST Countries v5), GeoJSON local. Experiência: 4 rotas; painel de país flutuante; cards com busca e filtros. Deploy: GitHub conectado à Vercel, Preview em branches e produção pela branch principal. Segurança: chave somente no servidor; cache de resposta no máximo 3 dias; build e testes antes de publicar.

Documento de planejamento, não substitui os termos dos serviços nem a documentação técnica. O protótipo do Stitch é uma referência visual; os comportamentos descritos precisam ser implementados e testados.